Sibyl: Technical Thesis

Sibyl

14 October 2026

CONTENTS

Sibyl technical thesis 3

The short answer 3

1. Is this observability or predictive intelligence? 3

2. What is the technical differentiation, concretely? 5

3. How much recovery are we responsible for? 6

4. Is there a data moat? 7

5. The stronger direction: controlling actions, not just failures 8

6. Where our advantage actually is 10

7. Where language models fit: Sibyl Sentinel (built, advisory) 10

8. External monitor agents: an open ecosystem, a deterministic core (built) 12

Decision and next build 13

Sibyl technical thesis

As of 2026-10-14, release 1.0.0. An honest answer to the review of Sibyl's positioning: what the built system does, what it does not, and what we build next.

The short answer

Sibyl should be **infrastructure that lets companies give AI agents more autonomy safely**. Predicting infrastructure failure is a supporting capability, not the thesis.

The smallest technically differentiated product is the **Action Gate**: an independent control point that sees an agent's proposed action before it executes and allows, blocks or escalates it, with signed policy and verifiable evidence. It is now built; its harness scenarios are in progress.

1. Is this observability or predictive intelligence?

Today it is mostly governed reliability, not predictive intelligence. We should say so.

Capability	How it works today	Comparable to existing tools?
Health score	Rule penalties over 1-minute rollups of gateway/SDK traffic	Yes, observability tools compute similar signals
”Prediction” (Phase 1)	Leading-indicator rules: EWMA trends, CUSUM change detection, rate-limit and budget proximity, provider status, combined with noisy-OR into probability bands	Better than static thresholds, but it is statistics, not learned prediction
Model drift detection	Served-model and fingerprint changes against a baseline; behaviour shifts by PSI and Jensen–Shannon over a frozen baseline per agent and model, with a sampling noise floor and hysteresis; watch-only alerts	Partly: some tools chart these signals. Deterministic statistics, not prediction: it detects a change after it happens
Plug-and-play monitoring	One line or zero code instruments OpenAI, Anthropic and Google Gen AI clients and LangChain/LangGraph, OpenAI Agents SDK and CrewAI; multi-agent run trees; unknown agents registered as discovered	Yes for the telemetry. The difference is that framework tools also pass the Action Gate, and discovered agents are never routed until confirmed
External monitor agents	The customer’s own Claude, OpenAI, Gemini, MCP or script monitor reads fleet metadata with a scoped key and posts advisory findings; signed webhooks push events out	Partly: the reading is ordinary. The difference is that the findings land beside a deterministic decision they cannot make
Action Gate	Allow, block or hold a proposed tool call from signed policy, at the gateway and in the SDK	**No.** Monitoring cannot see a healthy agent about to do something forbidden
Automatic failover by risk tier	Saga with approvals, undo, hold, kill switch, signed policy	**No.** Observability tools stop at alerting

Capability	How it works today	Comparable to existing tools?
Human handoff of unfinished work	Structured packet with checkpoint, steps, side effects	**No**
Tamper-evident evidence	Per-tenant hash chain, signed anchors, offline verification	**No**

What Sibyl does that observability does not is **act, under governance, and prove it**. That is the defensible part today. The prediction claim must be earned: the Phase 2 learned model ships only if the `sibyl-sim` harness shows it beats the rules on precision, recall and lead time, measured on each tenant's verdict-labelled alerts.

2. What is the technical differentiation, concretely?

Sibyl is not "an agent watching agents". No LLM sits in the detection or decision path. Agents may watch agents around Sibyl, Sentinel and the customer's own monitors (sections 7 and 8), but only as advisors to a person.

- **Where the information comes from.** Two in-path sources: a gateway that every model call passes through (OpenAI-compatible, native Anthropic Messages and native Gemini), and an SDK that reports model calls, agent runs and run trees, tool calls, handoffs, checkpoints and outcomes. Both are scrubbed of secrets before storage.
- **How detection works.** Deterministic rules over rollups (error rate, consecutive failures, latency, refusals, truncation, malformed tool calls, tool loops, provider status), with hysteresis so alerts don't chatter. Every score shows its arithmetic.

- **How drift detection works.** Each agent and model gets a frozen baseline (at least 500 calls over 6 hours). Every 10 minutes the last hour is compared with it: PSI for token counts and latency, Jensen–Shannon for refusal, truncation, tool-call and schema-failure shares, error mix and tool sequences. A noise floor is subtracted before the thresholds apply. A new served model or fingerprint raises `model_change`; two drift verdicts in a row raise `silent_drift`. Both are watch-only: they cannot start a failover or move the score. This is detection, not prediction.
- **How prediction works.** Phase 1 combines trend and change-point statistics into a probability band. This is a better warning threshold, not a forecast. Phase 2 is a per-tenant gradient-boosted model trained on operator verdicts. It runs in shadow mode and is promoted only past a measured gate (≥ 300 verdicts, beats rules on every tier gate).
- **How intervention is made safe.**
  - The gateway and SDK act only on Ed25519-signed policy snapshots.
  - Tier-1 agents need human approval for risky moves (four-eyes).
  - Every automated move can be undone, which holds automation for 30 minutes.
  - A single weak signal never triggers automation.
  - The kill switch beats everything.
  - Every decision is in the hash chain.
- **What improves with learning.** Per-tenant calibration of thresholds and, in Phase 2, of the risk model, from operator verdicts on alerts. Improvement is measured as precision, recall, lead time and false-failover rate on the harness and on real verdicts. If it does not improve, we keep the rules.

3. How much recovery are we responsible for?

We draw the line explicitly:

Sibyl owns	The customer's infrastructure owns
Which model, provider or agent a call goes to, and when	Business transaction integrity
Stopping traffic (kill switch, quarantine)	Idempotency of their tools and APIs
Handing unfinished work to a person, with context	Re-running or compensating side effects
Evidence of every decision	Their own data stores and backups

Shared dependencies. Switching between two agents that call the same failed upstream solves nothing. Sibyl already models this:

- The registry records dependency edges.
- Correlated failures on one provider collapse into a single provider incident.
- The failover viability check refuses targets that share the failed provider.

When no independent target exists, the right action is the human queue or a safe degraded mode, not a pointless switch. We should state this in the product and test it in the harness.

Duplicate actions. Locating where an invoice agent failed does not make it safe to resume. Sibyl's rule:

- A task whose recorded steps include external side effects is **never auto-resumed**. It goes to a person with the side-effect ledger, and the person decides.
- Automatic resume applies only to steps the agent declared idempotent.
- This is enforced in code (A-4): a handoff task or saga resume with a non-idempotent recorded side effect is held for the operator with its side-effect ledger.

4. Is there a data moat?

Not yet. Execution histories alone are not intelligence. What could become proprietary:

1. **A verdict-labelled incident corpus per tenant.** Every alert closed with a verdict is a label. Its value must be shown as measurable lift in the tenant's own model over time.
2. **The scenario library.** The `sibyl-sim` failure scenarios and their ground truth are reusable across customers without sharing customer data.
3. **Policy and continuity templates** per industry and workflow, learned from what customers configure and approve.

Data isolation. Decision D37 forbids pooling signals or models across tenants. Cross-deployment learning would need one of these:

- opt-in aggregate priors (counts and quantiles only, no content)
- federated training
- shipping only scenario and template improvements

Each needs a contractual clause and a technical boundary we have not built. Until then we do not claim a data moat.

5. The stronger direction: controlling actions, not just failures

There is a fundamental difference between:

- **A.** detecting that an agent is about to *fail*, and
- **B.** detecting that an agent is about to *do something it should not*.

An agent can be perfectly healthy and still prepare to transfer $100,000 when it is authorised for $10,000. Infrastructure monitoring cannot see that. An independent control layer can.

The Action Gate. Evaluate every proposed action before it executes, and **allow, block or escalate**:

- **Where it sits.**
  - The gateway already sees each model response before the agent acts on it, including the tool calls the model proposes.
  - The SDK's `@sibyl.tool` wrapper runs immediately before each tool executes.
  - Both are enforcement points we already ship.
- **What it checks.** Declarative, signed policy per agent and tool:
  - argument constraints, such as `amount <= 10000` or `destination in allowlist`
  - cumulative caps per period
  - tool allowlists
  - required dual control above a threshold
  - data-classification rules, such as no PII to an external tool
- **What happens on escalate.** The action is held. A person approves or denies it in the existing approval and queue flow, with a timeout that defaults to deny. Everything goes in the hash chain.
- **Why it is defensible.**
  - It is independent of the model and of the agent's own code.
  - It is enforced from signed policy at two points.
  - It is reversible and auditable.
  - It does not depend on proving a prediction claim.

Status: built. The evaluator, signed policies, enforcement at the gateway (OpenAI, Anthropic and Gemini protocols) and in the SDK (including tools run by LangChain, the OpenAI Agents SDK and CrewAI), holds with four-eyes and on-behalf-of, and the console screens on the live API. The harness scenarios (A-5) are in progress.

Built on and reused:

- the in-path gateway
- the SDK tool wrapper
- signed policy snapshots
- approvals with four-eyes
- the operator queue
- the kill switch

- the audit chain
- the console
- the scenario harness, which already includes a prompt-injection scenario

Research direction, not a claim. Predict risky behaviour earlier from the execution trajectory: the sequence of tool calls, their arguments and their drift from the agent's normal behaviour. Internal model signals are rarely accessible for hosted models, so trajectories are the practical input. This is a research bet, gated by the harness like everything else.

6. Where our advantage actually is

A startup does not need a novel algorithm. Our advantages, stated plainly:

1. **Enforcement in the path.** One base-URL change puts the gateway in front of the agent, with no code change. One line of SDK code, or none with `sibyl-run`, adds run trees and tool-level control.
2. **Governance that holds up in an audit.** Signed policy, approvals, undo, kill switch, and a hash chain that verifies offline.
3. **Execution:** deployability (self-hosted, one command, a public demo, and a demo and a real deployment each one link away), safety defaults and integration breadth.
4. **An open edge around a closed core.** Any agent the customer trusts can watch the fleet and advise (section 8); only the deterministic core can act. We do not have to win the "best reviewer model" race to win the control point.

We claim prediction only where the harness shows it.

7. Where language models fit: Sibyl Sentinel (built, advisory)

Customers want an agent that watches their other agents, using Claude, GPT or Gemini. We now offer one: **Sibyl Sentinel**, an optional, opt-in monitoring agent. It runs on the customer's own Anthropic, OpenAI or Google Gemini key, through one provider-neutral adapter. The key is sealed in the tenant vault and write-only; calls go only to the provider's API host.

It explains incidents, reviews held actions for the approver, flags semantic drift (purpose drift, injection-like tool sequences) that statistics cannot see, and writes a daily fleet summary.

It is **advisory and outside the enforcement path**:

- It cannot allow, block, approve, fail over or change policy. It never calls an enforcement route, and tests check that a reviewed hold stays pending and that a Sentinel alert never starts a failover.
- Its findings can open watch-only alerts and attach recommendations; a person or a deterministic rule decides.
- Agent data reaches the model as quoted, untrusted evidence, never as instructions; answers are structured and validated.
- It respects capture mode: metadata-only agents send no prompts or outputs.
- A daily budget is reserved before each call, so it is not overshot.
- Every finding records the provider, model, prompt hash, tokens and cost.
- **Its judgement is not yet measured.** Its behaviour is checked on the harness (S-33: it never enforces and keeps to budget and capture mode); its judgement is measured before anyone relies on it or it is offered as on by default.

That is our answer to “an agent watching agents is not a breakthrough”. The breakthrough claim rests on independent, signed, deterministic enforcement. Language models add judgement and explanation on top, where being wrong costs a recommendation, not a transfer.

Framework reach is built. The gateway speaks OpenAI-compatible, native Anthropic Messages and native Gemini, with fallback inside one protocol. The SDK instruments the OpenAI, Anthropic and Google Gen AI clients and LangChain/LangGraph, the OpenAI Agents SDK and CrewAI in one line or none, and gates their tools. Google ADK and AutoGen connectors are planned; any framework can also report OpenTelemetry GenAI spans.

8. External monitor agents: an open ecosystem, a deterministic core (built)

Decision D48 states the thesis: infrastructure that lets companies safely give agents more autonomy. Sentinel showed that language-model judgement fits that thesis when it advises and never decides. Wave 4 (D53) takes the same rule one step further: **any** agent can be the advisor, not only ours.

Why this belongs in the thesis.

- **Customers already have reviewers.** Teams run Claude assistants, OpenAI Agents SDK apps, Gemini pipelines and scripts that watch their agents. Asking them to replace those with Sentinel is a fight we do not need. Letting them plug in makes Sibyl the place where every opinion about an agent meets the decision it concerns.
- **More autonomy needs more eyes, not more deciders.** The more authority an agent has, the more reviewers its owners want. Every extra reviewer that could also act would be one more thing to secure, test and audit in the enforcement path. Reviewers that can only advise add judgement without adding risk to that path.
- **It keeps the claim honest.** Sibyl's guarantee is deterministic enforcement from signed policy. It does not depend on any model, ours or the customer's, being right. A monitor that is wrong, or prompt-injected through the agent data it reads, produces a wrong recommendation that a person reads and can rate as wrong. It cannot produce a wrong transfer.

How the boundary is enforced, not just promised.

- A monitor key holds two scopes, `monitor:read` and `findings:write`, and the database refuses any other combination for a monitor-bound key.
- An API key can use only the scopes a route lists for keys, and no route that allows, blocks, approves, fails over or changes policy lists either monitor scope.
- Monitor reads are limited to the monitor's watched agent groups and to metadata: no prompts, outputs or held-action arguments.
- A monitor's alerts are watch-only (cap 0, kind `leading`); the saga reads only detected alerts, so a monitor cannot start a failover. Its advice on a held action is shown to the approver, and the hold stays pending until a person or the timeout decides.

- Everything a monitored agent wrote reaches the monitor marked as untrusted data, and every registration, finding, rating and failed webhook delivery is in the audit chain.

How it connects. The Sibyl MCP server (`sibyl-mcp`, 13 tools, stdio or loopback HTTP) makes any MCP client a monitor; the SDK's `sibyl.monitor` client serves Gemini and Python agents; signed, retried webhooks (HMAC-SHA256 with a timestamp, connect-time egress checks) push alert, incident, held-action, drift and failed-run events out; plain REST serves scripts. Findings share Sentinel's shape and appear beside it, so a held payout can carry Sentinel's review and the customer's Claude reviewer's side by side while the approver decides.

What it does not change. No language model, ours or a customer's, sits in any detection, decision or enforcement path. Drift detection stays statistics. The Action Gate stays a pure function over signed policy.

What we do not claim. Sibyl does not measure an external monitor's quality; people rate findings. Monitor reads are metadata only today, so semantic review of content is Sentinel's job within the capture mode until a content path for monitors is designed under the same controls. Monitor keys have no live stream (webhooks and polling), and `run.failed` is the only run event.

Two ways to run (D55). The thesis is easier to test than to read. A demo with dummy data (`make demo`, or the demo Codespaces link) shows every screen, monitors included; the real deployment (`make up`, or the real Codespaces link) runs the full stack for real agents with OIDC sign-in and TLS. A public demo runs at https://sibyl-demo.onrender.com.

Decision and next build

Thesis: Sibyl lets companies give agents more autonomy safely. It controls actions before they execute and keeps agents running through failures, with evidence for every decision.

Smallest differentiated product: the Action Gate on top of the current platform.

#	Backlog item	Built on
A-0	Status 2026-10-12: evaluator, signed snapshot policies, gateway enforcement on all three protocols, SDK enforcement including framework tools, approval routes with on-behalf-of and self-approval refusal, decision history, held-action and counter workers, console screens on the live API, the A-4 guard and the harness scenarios (A-5: no unauthorised action executed in the Action Gate scenarios; full gate suite: 20 pass, 12 wait on rulings or unbuilt features) **built**. Also built in wave 3: plug-and-play monitoring and run trees, model drift detection (watch-only), Sibyl Sentinel (advisory). Built in wave 4 (release 1.0.0, 2026-10-14): external monitor agents (advisory), a runs list, demo and real deployments with Codespaces links, GitHub Actions CI and release images, tenant-tunable drift thresholds (API), the harness gate suite running (20 scenarios pass) and CI under 10 minutes	—
A-1	Action policy schema (tool, argument predicates, caps, dual control, data class) in signed snapshots	Policy snapshot, schemas
A-2	Enforcement at the gateway (proposed tool calls in responses) and in the SDK (`@sibyl.tool` pre-call)	Gateway, SDK
A-3	Escalation: hold the action, approve or deny in the queue, timeout defaults to deny, audit	Approvals, queue, audit
A-4	Never auto-resume a task with recorded external side effects	Handoff, saga
A-5	Harness scenarios: over-limit transfer, exfiltration via tool, injection-driven action, cumulative cap; gates: 0 unauthorised actions executed	`sibyl-sim`
A-6	Console: action policies, held actions, decision records	Console

What we stop claiming until measured: “predictive intelligence” and “data moat”. They become goals with explicit measurement plans (sections 1 and 4). Drift detection is described as detection, never prediction, and Sentinel carries no quality claim until

the harness measures it. External monitors carry no quality claim at all, and are never described as able to stop, approve or control anything.